LAB MANUAL
PART A
(PART A: TO BE REFERRED BY STUDENTS)
Experiment No: -9

Aim:- Implementation and analysis of the 15-Puzzle problem using the Branch and Bound technique.
Resource Required:- Any C compiler, VSCode or online compiler (e.g., OnlineGDB, Code::Blocks)

Theory:-
Branch and Bound Approach:

1. Branch and Bound is an optimization technique that systematically explores a state-space tree by branching into possible choices and using a bound to discard states that cannot lead to a better solution.
1. For the 15-Puzzle, each node represents a board configuration. The blank tile can move up, down, left, or right to generate child nodes.

15-Puzzle Problem:

1. The 15-Puzzle consists of a 4 x 4 board containing tiles numbered 1 to 15 and one blank space. The objective is to reach the goal arrangement by sliding tiles into the blank space.
1. A state is expanded by moving the blank tile to a valid adjacent position. Branch and Bound selects the most promising state according to a cost function.
1. Let g(n) be the number of moves made from the initial state and h(n) be the Manhattan-distance estimate of the remaining cost.
1. The bound or priority value is f(n) = g(n) + h(n). The live node having the minimum f(n) is expanded first.

Cost Function:

g(n) = depth of node n (number of moves from the initial state)
h(n) = sum of Manhattan distances of all numbered tiles from their goal positions
f(n) = g(n) + h(n)

Algorithm:-

```
Solve15Puzzle(initial, goal)
   create a priority queue of live nodes ordered by minimum f = g + h
   compute h(initial) and insert the initial node with g = 0
   while the priority queue is not empty
      remove the node having minimum f value
      if current board is the goal board
         print the path from initial state to goal state and stop
      locate the blank tile
      generate every valid child by moving the blank up, down, left, or right
      for each child, compute g = parent.g + 1, h = ManhattanDistance(child)
      set f = g + h and insert the child into the priority queue
```

C Implementation:-

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define N 4
#define MAX 10000

typedef struct Node {
    int mat[N][N];
    int x, y;
    int g, h, f;
    struct Node *parent;
} Node;

Node *pq[MAX];
int pqSize = 0;

int manhattan(int mat[N][N]) {
    int h = 0;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            int v = mat[i][j];
            if (v != 0) {
                int gi = (v - 1) / N;
                int gj = (v - 1) % N;
                h += abs(i - gi) + abs(j - gj);
            }
        }
    }
    return h;
}

int isGoal(int mat[N][N]) {
    int expected = 1;
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) {
            if (i == N - 1 && j == N - 1) return mat[i][j] == 0;
            if (mat[i][j] != expected++) return 0;
        }
    return 1;
}

void push(Node *n) {
    int i = pqSize++;
    while (i > 0 && pq[(i - 1) / 2]->f > n->f) {
```

```
            pq[i] = pq[(i - 1) / 2];
            i = (i - 1) / 2;
        }
        pq[i] = n;
    }

    Node *pop() {
    if (pqSize == 0) return NULL;
          Node *root = pq[0], *last = pq[--pqSize];
          int i = 0;
          while (2 * i + 1 < pqSize) {
              int c = 2 * i + 1;
        if (c + 1 < pqSize && pq[c + 1]->f < pq[c]->f) c++;
          if (last->f <= pq[c]->f) break;
          pq[i] = pq[c]; i = c;
       }
       if (pqSize > 0) pq[i] = last;
    return root;
  }

void printBoard(int a[N][N]) {
    for (int i = 0; i < N; i++) {
                                        for (int j = 0; j < N; j++) printf("%2d ", a[i][j]);
                                                                    printf("\n");
    }
    printf("\n");
  }

  Node *newNode(int mat[N][N], int x, int y, int nx, int ny, int g, Node *parent) {
    Node *n = (Node *)malloc(sizeof(Node));
    memcpy(n->mat, mat, sizeof(n->mat));
    int t = n->mat[x][y]; n->mat[x][y] = n->mat[nx][ny]; n->mat[nx][ny] = t;
    n->x = nx; n->y = ny; n->g = g; n->h = manhattan(n->mat); n->f = n->g + n->h; n->parent = parent;
    return n;
}

void printPath(Node *n) {
    if (n == NULL) return;
    printPath(n->parent);
    printBoard(n->mat);
}

int main() {
    int start[N][N] = {{1,2,3,4},{5,6,7,8},{9,10,11,12},{13,14,0,15}};
```

```
    Node *root = (Node *)malloc(sizeof(Node));
    memcpy(root->mat, start, sizeof(start));
    root->x = 3; root->y = 2; root->g = 0; root->h = manhattan(start); root->f = root->h; root->parent =
NULL;
    push(root);
    int dx[] = {-1,1,0,0}, dy[] = {0,0,-1,1};
    while (pqSize) {
        Node *cur = pop();
        if (isGoal(cur->mat)) {
            printf("Solution found in %d move(s):\n\n", cur->g);
            printPath(cur);
            return 0;
        }
        for (int k = 0; k < 4; k++) {
            int nx = cur->x + dx[k], ny = cur->y + dy[k];
            if (nx >= 0 && nx < N && ny >= 0 && ny < N) {
                if (cur->parent && nx == cur->parent->x && ny == cur->parent->y) continue;
                push(newNode(cur->mat, cur->x, cur->y, nx, ny, cur->g + 1, cur));
            }
        }
    }
    printf("No solution found.\n");
    return 0;
}
```

OUTPUT:

Solution found in 1 move(s):

Initial State: 1 2 3 4 | 5 6 7 8 | 9 10 11 12 | 13 14 0 15

Goal State: 1 2 3 4 | 5 6 7 8 | 9 10 11 12 | 13 14 15 0

Dry Run Example — Initial blank at row 4, column 3:

1. For the initial state, g = 0 and h = 1 because tile 15 is one position away from its goal position.
1. The blank can move up, left, or right. Branch and Bound generates these child states and computes f = g + h for each one.
1. Moving the blank right produces the goal state with g = 1, h = 0, and f = 1.
1. Since this live node has the minimum bound and satisfies the goal test, the solution path is printed.

Step	Move	g	h	f	Row 1	Row 2	Row 3	Row 4
0	Start	0	1	1	1 2 3 4	5 6 7 8	9 10 11 12	13 14 0 15
1	Blank Up	1	2	3	1 2 3 4	5 6 7 8	9 10 0 12	13 14 11 15
1	Blank Left	1	2	3	1 2 3 4	5 6 7 8	9 10 11 12	13 0 14 15
1	Blank Right	1	0	1	1 2 3 4	5 6 7 8	9 10 11 12	13 14 15 0

Complexity Analysis:

1. In the worst case, the search may explore a very large number of puzzle states; therefore the time complexity is exponential in the solution depth. The bound function greatly reduces unnecessary exploration in typical cases.

1. Space complexity is also exponential in the worst case because Branch and Bound stores multiple live nodes in the priority queue together with their parent links.

Algorithm	Best Case	Average Case	Worst Case
15-Puzzle (Branch and Bound)	O(1)	Depends on explored states	Exponential

Conclusion:

You should have learned how Branch and Bound is used to solve the 15-Puzzle, generated valid board states, evaluated nodes using g(n), h(n), and f(n), traced the solution path, and analyzed its time and space requirements.

PART B

(PART B: TO BE COMPLETED BY STUDENTS)

Roll No.	C-60	**Name:**	Avdhut Madhavi
Class	COMP/C	**Batch:**	C2
Date of Experiment:		**Date of Submission:**	
Grade:			

(Students must submit the soft copy as per the following segments within two hours of the practical. The soft copy must be uploaded on the Blackboard or emailed to the concerned lab in-charge faculty at the end of the practical in case there is no Blackboard access available.)

B.1: Implement and trace the following programs. Write the code, compile/run it, and record the output after each task.

1. Run the program using a different solvable 15-Puzzle configuration that requires at least 3 moves. Record the generated states and final solution path.

```
#include <stdio.h>
#include <stdlib.h>

#define N 4
#define MAX 1000

typedef struct {
    int board[16];
    int parent;
```

```
    int depth;
} Node;

Node nodes[MAX];
int count = 0;

int goal[16] = {
    1, 2, 3, 4,
    5, 6, 7, 8,
    9, 10, 11, 12,
    13, 14, 15, 0
};

void printBoard(int board[]) {
    int i;

    for (i = 0; i < 16; i++) {
        if (board[i] == 0)
            printf("    ");
        else
            printf("%2d ", board[i]);

        if ((i + 1) % 4 == 0)
            printf("\n");
    }
    printf("\n");
}

int isGoal(int board[]) {
    int i;

    for (i = 0; i < 16; i++) {
        if (board[i] != goal[i])
            return 0;
    }

    return 1;
}

int main() {
    int initial[16] = {
        1, 2, 3, 4,
        5, 6, 7, 8,
        9, 10, 11, 12,
```

```
    0, 13, 14, 15
};

int front = 0, rear = 0;
int i, j, blank, newBlank;
int moves[4] = {-4, 4, -1, 1};
int solution[MAX], pathCount = 0;
int current, found = -1;

for (i = 0; i < 16; i++)
    nodes[0].board[i] = initial[i];

nodes[0].parent = -1;
nodes[0].depth = 0;

count = 1;
rear = 1;

printf("Initial State:\n");
printBoard(initial);

while (front < rear) {
    current = front++;

    if (isGoal(nodes[current].board)) {
        found = current;
        break;
    }

    blank = 0;

    while (nodes[current].board[blank] != 0)
        blank++;

    for (i = 0; i < 4; i++) {
        newBlank = blank + moves[i];

        if (newBlank < 0 || newBlank >= 16)
            continue;

        if (i == 2 && blank % 4 == 0)
            continue;

        if (i == 3 && blank % 4 == 3)
```

```
            continue;

        if (rear >= MAX)
            break;

        for (j = 0; j < 16; j++)
            nodes[rear].board[j] =
                nodes[current].board[j];

        nodes[rear].board[blank] =
            nodes[rear].board[newBlank];

        nodes[rear].board[newBlank] = 0;
        nodes[rear].parent = current;
        nodes[rear].depth = nodes[current].depth + 1;

        printf("Generated State %d:\n", rear);
        printBoard(nodes[rear].board);

        rear++;
    }
}

if (found == -1) {
    printf("Solution not found.\n");
    return 0;
}

current = found;

while (current != -1) {
    solution[pathCount++] = current;
    current = nodes[current].parent;
}

printf("\nFinal Solution Path:\n");

for (i = pathCount - 1; i >= 0; i--) {
    printf("Step %d:\n",
           pathCount - 1 - i);
    printBoard(nodes[solution[i]].board);
}

printf("Total moves: %d\n", pathCount - 1);
```

```
    getch();
    return 0;
}
```

```
 1  2  3  4
 5  6  7  8
 9 10 11 12
   13 14 15

Step 1:
 1  2  3  4
 5  6  7  8
 9 10 11 12
13    14 15

Step 2:
 1  2  3  4
 5  6  7  8
 9 10 11 12
13 14    15

Step 3:
 1  2  3  4
 5  6  7  8
 9 10 11 12
13 14 15

Total moves: 3
_
```

2. Modify the initial board using another solvable configuration of your choice and verify the result by tracing g(n), h(n), and f(n) values.

For this experiment, we will use the A search algorithm* to solve the 15-Puzzle problem.

- (g(n)): Actual cost, or number of moves made from the initial state.
- (h(n)): Estimated cost to reach the goal state, calculated using the Manhattan distance heuristic.
- (f(n)): Total estimated cost.

f(n)=g(n)+h(n)

3. Do the analysis of the 15-Puzzle problem using the Branch and Bound technique.

Aim

To analyze and solve the 15-Puzzle problem using the Branch and Bound technique by exploring possible states and selecting the state with the minimum cost.

Initial state

```
 1  2  3  4
 5  6  7  8
 9 10 11 12
13  0 14 15
```

Goal state

1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0

Here, 0 represents the blank space.

Algorithm

Branch and Bound explores possible moves and selects the most promising state based on its cost.
We use:

$$f(n) = g(n) + h(n)$$

Where:

- (g(n)): Number of moves made from the initial state.
- (h(n)): Manhattan distance heuristic, the estimated moves remaining to reach the goal.
- (f(n)): Total estimated cost.

Step-by-step analysis

Step 1: Initial state

1 2 3 4
5 6 7 8
9 10 11 12
13 0 14 15

Tiles 14 and 15 are each one position away from their goal positions.

$$g(n) = 0, \quad h(n) = 2$$
$$f(n) = 0 + 2 = \boxed{2}$$

Step 2: Move the blank right

1 2 3 4
5 6 7 8
9 10 11 12
13 14 0 15

Tile 14 is now in its correct position. Tile 15 remains one position away.

$$g(n) = 1, \quad h(n) = 1`$$
$$f(n) = 1 + 1 = \boxed{2}$$

Step 3: Move the blank right again

1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0

The goal state is reached.

$$g(n) = 2, \quad h(n) = 0$$

$$f(n) = 2 + 0 = \boxed{2}$$

Cost table

State	Move	\(g(n)\)	\(h(n)\)	\(f(n)\)
Initial	None	0	2	2
State 2	Right	1	1	2
Goal	Right	2	0	2

Result and complexity analysis

The Branch and Bound technique finds the goal state in 2 moves by selecting the most promising states based on their estimated cost.

- Time complexity: Exponential in the worst case, commonly expressed as \(O(b^d)\), where \(b\) is the branching factor and \(d\) is the solution depth.
- Space complexity: Can also be exponential because generated states may need to be stored.
- Advantage: It explores promising states first and can avoid exploring paths whose costs exceed the current best solution.

Conclusion: Branch and Bound successfully solves the selected 15-Puzzle configuration in two moves using the cost function \(f(n)=g(n)+h(n)\).